```
const express = require("express");
const axios = require("axios");

const app = express();
app.use(express.json());

const TELEGRAM_TOKEN = process.env.TELEGRAM_TOKEN;
const ANTHROPIC_KEY = process.env.ANTHROPIC_KEY;
const GENIEMAP_TOKEN = process.env.GENIEMAP_TOKEN;
const TELEGRAM_API = "https://api.telegram.org/bot" + TELEGRAM_TOKEN;

const conversations = {};

async function sendMessage(chatId, text) {
  try {
    await axios.post(TELEGRAM_API + "/sendMessage", {
      chat_id: chatId,
      text: text,
      parse_mode: "Markdown",
    });
  } catch (e) {
    try {
      await axios.post(TELEGRAM_API + "/sendMessage", {
        chat_id: chatId,
        text: text.replace(/[*_`]/g, ""),
      });
    } catch (e2) {}
  }
}

async function sendTyping(chatId) {
  try {
    await axios.post(TELEGRAM_API + "/sendChatAction", {
      chat_id: chatId,
      action: "typing",
    });
  } catch (e) {}
}

async function genieMapFetch(endpoint, params) {
  if (!params) params = {};
  var url = "https://api.geniemap.net/v1" + endpoint;
  var queryParts = [];
  Object.keys(params).forEach(function(k) {
```

```
    if (params[k] !== undefined && params[k] !== "") {
      queryParts.push(k + "=" + encodeURIComponent(params[k]));
    }
  });
  if (queryParts.length > 0) url = url + "?" + queryParts.join("&");
  var res = await axios.get(url, {
    headers: {
      Authorization: "Bearer " + GENIEMAP_TOKEN,
      Accept: "application/json",
    },
    timeout: 10000,
  });
  return res.data;
}

var SYSTEM_PROMPT = "You are the Palladium Group Real Estate AI assistant, integr

var TOOLS = [
  {
    name: "get_areas",
    description: "Get list of UAE real estate areas from GenieMap.",
    input_schema: {
      type: "object",
      properties: {
        search: { type: "string", description: "Search term" },
        page: { type: "integer", description: "Page number" },
      },
    },
  },
  {
    name: "get_projects",
    description: "Get list of UAE real estate projects.",
    input_schema: {
      type: "object",
      properties: {
        search: { type: "string", description: "Search term" },
        page: { type: "integer", description: "Page number" },
      },
    },
  },
  {
    name: "get_neighborhoods",
    description: "Get list of UAE neighborhoods.",
    input_schema: {
      type: "object",
      properties: {
        search: { type: "string", description: "Search term" },
```

```
        page: { type: "integer", description: "Page number" },
      },
    },
  },
  {
    name: "get_project_detail",
    description: "Get detailed info about a specific project by ID.",
    input_schema: {
      type: "object",
      properties: {
        id: { type: "integer", description: "Project ID" },
      },
      required: ["id"],
    },
  },
  {
    name: "get_area_detail",
    description: "Get detailed info about a specific area by ID.",
    input_schema: {
      type: "object",
      properties: {
        id: { type: "integer", description: "Area ID" },
      },
      required: ["id"],
    },
  },
];

async function executeTool(toolName, toolInput) {
  try {
    if (toolName === "get_areas") {
      return await genieMapFetch("/areas", { search: toolInput.search, page: tool
    } else if (toolName === "get_projects") {
      return await genieMapFetch("/projects", { search: toolInput.search, page: t
    } else if (toolName === "get_neighborhoods") {
      return await genieMapFetch("/neighborhoods", { search: toolInput.search, pa
    } else if (toolName === "get_project_detail") {
      return await genieMapFetch("/projects/" + toolInput.id);
    } else if (toolName === "get_area_detail") {
      return await genieMapFetch("/areas/" + toolInput.id);
    }
    return { error: "Unknown tool" };
  } catch (err) {
    return { error: err.message, note: "GenieMap API may require IP whitelisting"
  }
}
```

```
async function askClaude(chatId, userMessage) {
  if (!conversations[chatId]) conversations[chatId] = [];
  conversations[chatId].push({ role: "user", content: userMessage });
  if (conversations[chatId].length > 20) {
    conversations[chatId] = conversations[chatId].slice(-20);
  }

  var messages = conversations[chatId].slice();

  for (var i = 0; i < 5; i++) {
    var response = await axios.post(
      "https://api.anthropic.com/v1/messages",
      {
        model: "claude-sonnet-4-20250514",
        max_tokens: 2000,
        system: SYSTEM_PROMPT,
        tools: TOOLS,
        messages: messages,
      },
      {
        headers: {
          "x-api-key": ANTHROPIC_KEY,
          "anthropic-version": "2023-06-01",
          "Content-Type": "application/json",
        },
        timeout: 30000,
      }
    );

    var data = response.data;
    messages.push({ role: "assistant", content: data.content });

    if (data.stop_reason === "end_turn") {
      var textBlock = data.content.find(function(b) { return b.type === "text"; }
      conversations[chatId] = messages;
      return textBlock ? textBlock.text : "Done.";
    }

    if (data.stop_reason === "tool_use") {
      var toolUseBlocks = data.content.filter(function(b) { return b.type === "to
      var toolResults = [];
      for (var j = 0; j < toolUseBlocks.length; j++) {
        var toolUse = toolUseBlocks[j];
        var result = await executeTool(toolUse.name, toolUse.input);
        toolResults.push({
          type: "tool_result",
          tool_use_id: toolUse.id,
```

```
          content: JSON.stringify(result),
        });
      }
      messages.push({ role: "user", content: toolResults });
      continue;
    }
    break;
  }

  conversations[chatId] = messages;
  return "I processed your request. Please try again.";
}

app.post("/webhook", async function(req, res) {
  res.sendStatus(200);
  try {
    var update = req.body;
    if (!update.message || !update.message.text) return;

    var chatId = update.message.chat.id;
    var text = update.message.text;
    var firstName = update.message.from ? update.message.from.first_name : "there

    if (text === "/start") {
      await sendMessage(chatId, "Welcome " + firstName + "! I am the Palladium Pr
      return;
    }

    if (text === "/clear") {
      conversations[chatId] = [];
      await sendMessage(chatId, "Conversation cleared!");
      return;
    }

    if (text === "/help") {
      await sendMessage(chatId, "Commands:\n/start - Welcome\n/clear - Clear hist
      return;
    }

    await sendTyping(chatId);
    var reply = await askClaude(chatId, text);
    await sendMessage(chatId, reply);

  } catch (err) {
    console.log("Error: " + err.message);
  }
});
```

```
app.get("/", function(req, res) {
  res.send("Palladium Property Bot is running");
});

var PORT = process.env.PORT || 3000;
app.listen(PORT, function() {
  console.log("Bot running on port " + PORT);
});
```